

Instituto Tecnológico de Buenos Aires

Teoría de Lenguajes, Autómatas y Compiladores

Trabajo Práctico Especial

AÑO 2025



INFORME

Alumno:

· Rossi Nicolás - 53225

Fecha de entrega: 22/6/2025

Introducción	1
Modelo Computacional	2
Dominio	2
Lenguaje	2
Implementación	4
Frontend	4
Backend	5
Adicionales	6
Dificultades Encontradas	7
Futuras expansiones	8
Referencias	8

Introducción

En el presente informe se detalla el diseño, implementación y justificación técnica de un minificador de JavaScript desarrollado en C. El sistema implementa las etapas clásicas de un compilador para un subconjunto bien definido del lenguaje, aplicando reglas básicas de minificación semántica orientadas a reducir el tamaño del código sin modificar su comportamiento observable. (para más detalles ver [Especificación](#))

Modelo Computacional

Dominio

El dominio central del proyecto es la minificación semántica de código JavaScript. Si bien JavaScript es un lenguaje de propósito general, la problemática de la reducción del tamaño del código para su despliegue eficiente (especialmente en aplicaciones web) justifica la construcción de herramientas especializadas.

El minificador implementado aborda este dominio como un “procesador” de un subconjunto seleccionado del lenguaje JavaScript, permitiendo aplicar transformaciones tanto textuales como semánticas para reducir el tamaño final del código.

Estas transformaciones requieren una comprensión precisa de las semántica del lenguaje, ya que cualquier optimización debe preservar la semántica observable del programa original: es decir, el código minificado debe comportarse exactamente igual al original en cualquier entorno estándar de ejecución de JavaScript.

Lenguaje

El lenguaje procesado es un subconjunto definido de JavaScript, formalizado mediante una gramática del tipo $G = \langle \Sigma, N, \Pi, S \rangle$:

Σ (Alfabeto): Definido por el analizador léxico (Flex), abarca identificadores, palabras clave, operadores, literales y delimitadores del subset soportado.

N (No-terminales), Π (Producciones) y S (Símbolo inicial): Definidos en el analizador sintáctico (Bison). Suele tomar como símbolo inicial a `program` o un bloque raíz equivalente.

Las características soportadas incluyen:

- Eliminación de espacios en blanco y comentarios:
El lexer ignora estos elementos y no los incorpora al AST¹, haciendo su minificación “natural” y segura.
- Constant folding:
El parser y el análisis semántico evalúan expresiones constantes (ejemplo: `2 * 3` se reemplaza por `6`), reduciendo el trabajo en tiempo de ejecución y el tamaño del output.
- Renombrado seguro de variables:
Todos los identificadores son acortados, asegurando unicidad por scope. Esto se implementa a nivel de tabla de símbolos y generación de código.

[¹AST](#): Árbol de Sintaxis Abstracta (por sus siglas en inglés)

- Minificación de booleanos:
Los literales `true` y `false` se transforman en `!0` y `!1`, respectivamente, según las reglas de coerción booleana en JavaScript.
- Soporte para operadores lógicos, de comparación y bitwise:
Incluye `==`, `!=`, `===`, `!==`, `<`, `>`, `>=`, `<=`, `|`, `&`, `^`, `~`, entre otros.
- Soporte para estructuras de control:
Se soportan `for`, `if`, `while`, `do..while`, `switch`, `try..catch` (con sus variantes y sintaxis canónica).
- Concatenación de literales:
Expresiones como `"a" + "b"` son evaluadas en tiempo de compilación a `"ab"`.

La implementación de la gramática hace uso extensivo de atributos y valores semánticos mediante la directiva `%union` de Bison.

Cada token y no-terminal relevante transporta información estructurada, necesaria para construir el AST y transferir contexto entre fases (por ejemplo, tipos, valores, nombres minificados, etc).

La construcción del AST ocurre en las acciones semánticas de las producciones de Bison, pasando nodos y listas mediante el valor semántico correspondiente.

La precedencia y asociatividad de los operadores se define explícitamente en la gramática, utilizando las directivas `%left` y `%right` de Bison. Esto garantiza un parseo correcto y natural de expresiones aritméticas y lógicas, alineando el comportamiento con el estándar ECMAScript y evitando ambigüedades en la interpretación del código fuente.

Implementación

Frontend

El frontend constituye la primera etapa del compilador: recibe el código fuente en texto plano y lo transforma en una representación estructurada —el Árbol de Sintaxis Abstracta (AST)— que servirá de insumo para todas las fases posteriores. Para lograrlo se implementaron dos sub-módulos claramente diferenciados pero fuertemente acoplados: el scanner (léxico) y el parser (sintáctico).

El análisis léxico, construido con Flex, recorre secuencialmente el archivo de entrada y agrupa los caracteres en tokens significativos: identificadores, literales numéricos, cadenas, palabras clave, operadores y signos de puntuación. Cada token se empaqueta con la información mínima necesaria (lexema, tipo, y metadata) y se envía al analizador sintáctico. Gracias a la definición de expresiones regulares, el scanner descarta de forma temprana espacios en blanco, tabulaciones, saltos de línea y comentarios, lo que ya supone un primer paso hacia la minificación.

El análisis sintáctico se implementó en Bison a partir de una gramática diseñada para un subconjunto coherente de ECMAScript (ES2021). La gramática cubre:

- declaraciones (let/const, funciones con nombre),
- expresiones aritméticas, lógicas y de llamada,
- control de flujo (if/else, bucles for, while, do-while, bloques try/catch/finally),
- literales compuestos (arreglos),
- así como reglas auxiliares para precedencia y asociatividad que reducen los conflictos

Cada reducción en Bison invoca una acción semántica que crea y enlaza nodos del AST —estructuras definidas en [AbstractSyntaxTree.h](#)— de forma que la jerarquía resultante refleja exclusivamente la semántica del programa y no su forma textual.

El frontend, también, incorpora un mecanismo de reporte de errores tempranos: el scanner detecta caracteres ilegales; el parser informa construcciones sintácticas inválidas para el subconjunto del lenguaje seleccionado.

En resumen, el frontend genera una representación interna ya depurada de espacios y comentarios, asegurando una estructura válida y lista para ser validada semánticamente y transformada por el backend del minificador.

Backend

El backend toma el AST entregado por el frontend y lo recorre en dos pasadas claramente diferenciadas: análisis semántico y generación/minificación de código. Ambas etapas comparten infraestructura –la tabla de símbolos– pero persiguen objetivos distintos: la primera garantiza que el programa sea válido según las reglas del subconjunto de JavaScript elegido; la segunda produce la versión minificada, aplicando transformaciones que reducen el tamaño sin alterar la semántica.

Análisis semántico

Se implementó en [SemanticAnalyzer.c](#) mediante un recorrido recursivo “top-down” del AST.

- Para cada nodo se consulta o actualiza una tabla de símbolos basada en un stack de frames (uno por bloque léxico). Esta tabla se gestiona con operaciones `stEnterScope/stExitScope`, de modo se respeten las reglas de scope de JavaScript.
- El analizador detecta uso de identificadores antes de ser declarados y violaciones de la inmutabilidad de `const`.
- Se comprueban reglas tempranas del estándar (por ejemplo, `const` sin inicializador y parámetros duplicados en funciones).
- Se aplica constant folding: si ambos operandos de una operación binaria son literales, el nodo se reemplaza in-place por el literal resultante.

Minificación y generación de código

Una vez validado el AST, la fase de generación produce JavaScript minimizado:

- El archivo [ExpressionGenerator.c](#) contiene una tabla de función-puntero que despacha cada tipo de expresión a su emisor correspondiente. Esto evita largos switch y facilita extender el repertorio de nodos, además de proveer acceso en $O(1)$.
- Por practicidad, los booleanos literales se reescriben al momento de la generación de código
- El minificado de identificadores se apoya en la tabla de símbolos [SymbolTable.h](#):
Al insertar un nombre se le asigna, con `_nextMinifiedName()`, la siguiente combinación “base-26” (a, b, ..., z, aa, ...). Para posteriormente, durante la emisión, si un identificador cuenta con nombre minificado se emite éste; de lo contrario se usa su lexema original, garantizando que las referencias cruzadas sigan siendo coherentes.
- Para preservar la semántica con el menor número de caracteres, se omiten todos los espacios innecesarios. La rutina `output()` centraliza la escritura y facilita redirigir la salida a `stdout` o a un archivo (según la variable de `OUTPUT_FILE`).

Tabla de símbolos con hash djb2

Para la resolución de identificadores se implementó una tabla de símbolos basada en hashtable con djb2 (Daniel J. Bernstein²) como función de dispersión. Esta implementación ofrece un buen balance excelente entre velocidad y distribución uniforme de claves de texto.

- Cada frame contiene un arreglo `bucket[BUCKET_SIZE]` para manejar las colisiones por encadenamiento simple.
- La inserción `stInsert` evita duplicados en el ámbito actual y, además de la metadata semántica, almacena el nombre minificado.
- Los frames que salen de pila no se destruyen inmediatamente: se encadenan en la lista “zombies” para que el generador pueda seguir consultando sus nombres minificados. Al final de la compilación `stPurge` libera todos los zombies de una sola pasada, eliminando fugas sin comprometer la seguridad de memoria.

Adicionales

Test suite when/expect

Se desarrolló un script de pruebas automatizado [test-suite.sh](https://github.com/lukeed/test-suite) donde:

Etapa	Descripción
when	especifica la entrada de prueba (código JS a minificar)
expect	contiene la salida minificada

El script compila cada caso, captura stdout y compara contra el expect usando `diff -u` para validar que la salida del compilador sea lo esperada.

Esta etapa de test se agregó además al pipeline de Github Actions para mantener el proyecto en un estado válido durante el proceso de desarrollo.

Dificultades Encontradas

Conflictos shift/reduce en Bison

El primer gran obstáculo fue alcanzar un parse limpio. La gramática original producía conflictos shift/reduce (principalmente por la ambigüedad entre los operadores unarios/pós-fijo y la palabra reservada `new`). Se resolvió re-ordenando las prioridades con `%precedence` y creando un token ficticio `NEW_PREC` para separar el parse de `new foo()` vs `new foo.bar`.

Gestión de memoria (fugas y use-after-free)

El cambio de expresiones binarias a literales durante el constant folding invalidaba subárboles todavía referenciados. La primera versión sencillamente reasignaba el nodo, provocando fugas y, heap-use-after-free detectados por el analizador de memoria (ASan). Se solucionó añadiendo la utilidad `releaseExpression` en combinación con [_replaceWithLiteral](#), de modo que el contenido antiguo se libera de forma segura antes de reusar la estructura.

Gestión de memoria y zombies

Para que el generador pudiera seguir accediendo a nombres minificados fuera de frame, se intentó copiar cada símbolo al frame padre; esto ocasionó errores de duplicados y sobrescrituras en la memoria. La estrategia definitiva para solventarlo fue elaborar la lista de **zombies**: los frames que salen de pila se marcan para destrucción diferida.

Minificación de nombres en presencia de shadowing

Inicialmente se asignaba un identificador minificado globalmente. Pero, así, al encontrarse una variable con el mismo nombre en un bloque, se violaba la unicidad dentro del ámbito padre. Lo anterior fue solucionado con el rediseño de la función [_nextMinifiedName](#) para que opere por frame y se añadió un sufijo incremental cuando el nombre corto ya está ocupado en la cadena de ancestros.

Futuras expansiones

Debido al tiempo acotado para este trabajo, se logró apenas una minificación muy básica, por lo que aún hay un sin fin de extensiones posibles. Desde la capacidad para soportar un subset más extenso de JavaScript, como también optar por optimizaciones más complejas del lenguaje.

A continuación, se mencionan algunas expansiones posibles:

- Eliminación de código duplicado
- Eliminación de código inaccesible
- Minificación de propiedades y campos internos de objetos.
- Minificación de if-else por estructuras ternarias
- Reemplazar llamadas a funciones triviales por el cuerpo de sus funciones.
- Extender el subset de Javascript soportado.

Referencias

- CSE York University. (s. f.). *Hash Functions (djb2, sdbm, etc.)*. Recuperado el 23 de junio de 2025, de <http://www.cse.yorku.ca/~oz/hash.html>
- Ecma International. (2020). *ECMAScript 2020 language specification* (12th ed.). Recuperado de <https://tc39.es/ecma262/2020/#sec-ecmascript-language-lexical-grammar>
- Westes, J. O. (s. f.). *Flex: The fast lexical analyzer generator (Manual)*. Recuperado el 23 de junio de 2025, de <https://westes.github.io/flex/manual/>
- Aiken, A. (2012). *Introducing Bison* (Handout 120). Stanford University, Department of Computer Science. Recuperado de <https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/handouts/120%20Introducing%20bison.pdf>